07. Faux positifs dans les tests unitaires

Cette page complète les FAQ sur les tests unitaires SQL et sur le fonctionnement des déclencheurs différés. Elle explique pourquoi, dans un test **positif** (qui doit réussir, et donc sans `should_fail`), il est recommandé de terminer le scénario par `SET CONSTRAINTS ALL IMMEDIATE`.

Le problème : les faux positifs

Imaginons que vous ayez développé un cas de test **supposé positif**, mais que vous ayez fait une **erreur** dans le scénario et que vos commandes SQL ne respectent pas un invariant implémenté par un **déclencheur différé**.

Lors de l'exécution de ce test, la vérification de cet invariant ne sera effectuée qu'**à la fin de la transaction**, c'est-à-dire au moment du `COMMIT`. Or les tests positifs se terminent par un `ROLLBACK` qui annule non seulement les commandes SQL du scénario, mais aussi le déclenchement du déclencheur différé.

Du coup, le test va passer à tort sans que l'erreur ne soit détectée. C'est ce qu'on appelle un **faux positif**.

> Note : si votre scénario de test est correctement implémenté et respecte les invariants, le problème ne se pose pas, mais nul n'est à l'abri d'une erreur ou d'un oubli 😅

La solution

La commande `SET CONSTRAINTS ALL IMMEDIATE` force PostgreSQL à **vérifier tout de suite**, dans la transaction en cours, les déclencheurs différés. Si vous savez que votre scénario de test risque de déclencher une exception dans un trigger différé, il est recommandé d'appeler cette commande après les dernières commandes du scénario, avant la fin du bloc `DO`.

Par exemple :

```
begin;
do
$test$
    begin
        raise notice 'TEST: BR-xx';

        -- ... commandes SQL du scénario ...

        set constraints all immediate;
```

```
    end
$test$;
rollback;
```



Lien avec les tests négatifs (`should_fail`)

Si vous examinez le code de la fonction `should_fail`, vous observerez qu'elle exécute elle-même `SET CONSTRAINTS ALL IMMEDIATE` pour la même raison.